

HƯỚNG DẪN THỰC HÀNH: KHỞI TẠO MÔI TRƯỜNG & PHÁT TRIỂN SMART CONTRACT ĐẦU TIÊN

Giảng viên hướng dẫn: TS. Hà Ngọc Long

Đơn vị: Trường Đại học Kinh tế, Đại học Huế

Mục tiêu: Giúp sinh viên nắm vững kiến trúc cơ sở của Ethereum (EVM), thiết lập ví Web3 an toàn, kết nối môi trường phát triển Antigravity IDE với Remix, và triển khai thành công Smart Contract đầu tiên.

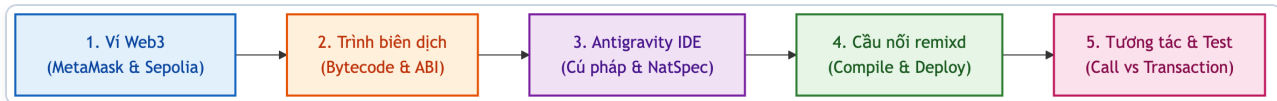
MỤC LỤC

- [1. Mục tiêu bài học & Chuẩn đầu ra](#)
- [2. Phần 1: Khởi tạo và bảo mật Ví Web3 \(MetaMask\)](#)
- [3. Phần 2: Bản chất Compiler & Các phương pháp chạy Remix trên máy tính](#)
- [4. Phần 3: Sử dụng Antigravity IDE viết Smart Contract đầu tiên](#)
- [5. Phần 4: Kết nối Antigravity với Remix, Compile & Deploy](#)
- [6. Phần 5: Bài tập thực hành & Câu hỏi tự đánh giá](#)

1. MỤC TIÊU BÀI HỌC & CHUẨN ĐẦU RA

Sau khi hoàn thành bài thực hành này, sinh viên có khả năng:

- * **Hiểu bản chất:** Phân biệt được Tài khoản cá nhân (EOA) và Tài khoản hợp đồng (Contract Account); hiểu cơ chế phí Gas và vai trò của EVM (Ethereum Virtual Machine).
- * **Thao tác an toàn:** Tự tạo và quản lý khóa cá nhân (Private Key) / Cụm từ khôi phục (Seed Phrase) theo đúng chuẩn an toàn thông tin mật mã.
- * **Làm chủ công cụ:** Sử dụng **Antigravity IDE** để viết code chuẩn mực, đồng thời tích hợp với **Remix IDE** để biên dịch, gỡ lỗi và triển khai.
- * **Triển khai thực tế:** Nắm vững quy trình từ mã nguồn Solidity → Biên dịch (Bytecode + ABI) → Triển khai trên môi trường giả lập (Remix VM) và mạng thử nghiệm công khai (Ethereum Sepolia Testnet).



Lộ trình thực hành 5 bước

PHẦN 1: KHỞI TẠO VÀ BẢO MẬT VÍ WEB3 (METAMASK)

1.1. Kiến thức nền tảng: Ví Web3 thực chất là gì?

⚠️ Quan niệm sai lầm phổ biến: "Ví lưu trữ coin/token của tôi bên trong điện thoại hay máy tính."

Thực tế kỹ thuật: Toàn bộ số dư và Smart Contract đều nằm trên **sổ cái phân tán (Distributed Ledger)** của **Blockchain**. Ví (Wallet) chỉ là một **công cụ quản lý khóa mật mã (Key Manager)** giúp bạn ký xác thực các giao dịch gửi lên mạng lưới.

- **Cặp khóa mật mã bất đối xứng (Asymmetric Key Pair):**
- **Public Key (Khóa công khai) → Địa chỉ ví (Address):** Dạng chuỗi hexa bắt đầu bằng `0x...` (ví dụ: `0x71C...392`). Bạn có thể công khai địa chỉ này để người khác gửi ETH hoặc gọi hợp đồng của bạn.
- **Private Key (Khóa riêng tư):** Chuỗi ký tự bí mật dùng để tạo chữ ký số (Digital Signature). Bất kỳ ai có khóa này đều có toàn quyền kiểm soát toàn bộ tài sản trong ví!
- **Secret Recovery Phrase (Cụm từ khôi phục bí mật / Seed Phrase):** 12 hoặc 24 từ tiếng Anh chuẩn chuẩn BIP-39, được dùng để sinh ra tập hợp tất cả các Private Key của bạn.

1.2. Các bước cài đặt MetaMask

1. Sử dụng trình duyệt Chrome, Brave hoặc Edge, truy cập trang chủ chính thức: <https://metamask.io/download/>.
2. Nhấn **Install MetaMask** để cài đặt tiện ích mở rộng (Extension).
3. Sau khi cài đặt, chọn **Create a new wallet** (Tạo ví mới).
4. Thiết lập mật khẩu mở khóa ứng dụng cục bộ (Local Password).

1.3. NGUYÊN TẮC AN TOÀN BẢO MẬT KHÓA (BẮT BUỘC TUÂN THỦ)

NGUYÊN TẮC AN TOÀN BẢO MẬT KHÓA

1. ****KHÔNG BAO GIỜ**** chụp màn hình, lưu file Word/Notepad, hoặc gửi 12 từ khôi phục qua Zalo, Messenger, Email, Google Drive. 2. Hãy chép 12 từ khóa ra sổ tay vật lý và cất giữ an toàn. 3. ****KHÔNG BAO GIỜ**** dán Private Key hay Mnemonic vào mã nguồn code rồi commit lên GitHub. Kẻ gian có bot quét tự động 24/7 và sẽ rút cạn tiền trong ví chỉ sau 3 giây!

1.4. Chuyển sang mạng thử nghiệm Ethereum Sepolia Testnet & Nhận ETH miễn phí

Để thực hành lập trình mà không mất tiền thật, chúng ta sử dụng **Mạng thử nghiệm Sepolia (Sepolia Testnet)**:

1. Mở tiện ích MetaMask → Click vào biểu tượng chọn mạng ở góc trên bên trái (mặc định là *Ethereum Mainnet*).

2. Tích chọn **Show test networks** (Hiện thị các mạng thử nghiệm).

3. Chọn mạng **Sepolia**. Địa chỉ ví của bạn vẫn giữ nguyên định dạng `0x...`.

4. **Nhận ETH Testnet (Faucet) miễn phí:**

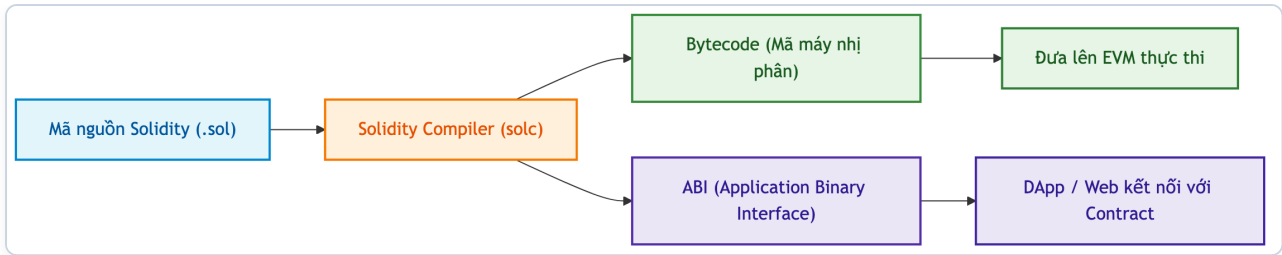
* Truy cập các trang vòi cấp phát ETH thử nghiệm (Google Search: *Sepolia Faucet* như `sepoliafaucet.com` hoặc `infura.io/faucet/sepolia`).

* Dán địa chỉ ví `0x...` vào để nhận từ 0.05 đến 0.5 Sepolia ETH dùng làm phí Gas thử nghiệm.

PHẦN 2: BÀN CHẤT COMPILER & CÁC PHƯƠNG PHÁP CHẠY REMIX TRÊN MÁY TÍNH

2.1. Bàn chất Trình biên dịch (Solidity Compiler - `solc`)

Máy ảo Ethereum (EVM) không hiểu được mã nguồn văn bản Solidity (`.sol`). Quá trình biên dịch sẽ chuyển hóa code thành 2 thành phần cốt lõi:



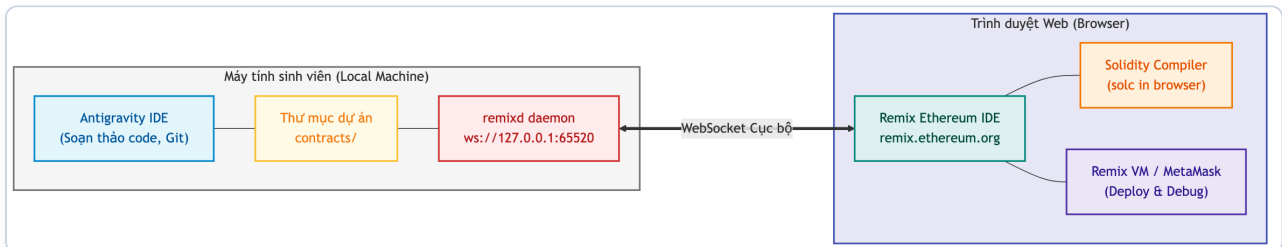
Quy trình biên dịch Solidity sang Bytecode và ABI

- **Bytecode:** Chuỗi mã máy hexa (ví dụ: `0x6080604052348015 ...`) sẽ được nạp trực tiếp vào trạng thái của mạng lưới khi deploy.
- **ABI (Application Binary Interface):** Một file định dạng JSON mô tả danh sách các hàm, kiểu dữ liệu tham số đầu vào/đầu ra, và các sự kiện (events). Frontend (React, Vue, Web3.js, Ethers.js) căn cứ vào ABI để biết cách gọi hàm trên hợp đồng.

2.2. Hai cách thiết lập Remix để làm việc trên máy tính

Cách 1: Kết hợp Antigravity IDE + Remix Web qua `remixd` (Khuyến nghị chuẩn thực hành)

Đây là quy trình chuyên nghiệp nhất được các blockchain engineer sử dụng: Sinh viên code và quản lý phiên bản (Git) trực tiếp trong **Antigravity IDE** trên máy tính, đồng thời sử dụng cầu nối **Remix Daemon (`remixd`)** để giao diện Remix trên trình duyệt kết nối trực tiếp vào thư mục mã nguồn nội bộ.



Mô hình kết nối Antigravity IDE với Remix qua remixd daemon

A. Chuẩn bị môi trường (Cài đặt Node.js)

Cầu nối `remixd` chạy trên nền Node.js. Sinh viên cần kiểm tra xem máy đã có Node.js hay chưa:

- * Mở Terminal và gõ: `node -v` và `npm -v`.
- * Nếu chưa có, tải bản **LTS (Long Term Support)** chính thức:
- * **Trên Windows:** Truy cập nodejs.org, tải file cài đặt `.msi` (64-bit). Khi cài đặt, giữ nguyên các tùy chọn mặc định (đặc biệt là mục *Add to PATH*), bấm *Next* cho đến khi hoàn tất.

* **Trên macOS:** Tải file `.pkg` từ nodejs.org hoặc cài nhanh qua Homebrew:

```
bash brew install node
```

B. Hướng dẫn chi tiết theo từng Hệ điều hành

Dành cho người dùng macOS:

1. **Mở Terminal trong Antigravity IDE:**
 2. Dùng phím tắt: `Control + `` (Control + phím dấu huyền cạnh số 1) hoặc vào menu `Terminal` → `New Terminal`.
 3. Shell mặc định sẽ là `zsh` hoặc `bash`.
 4. **Khởi chạy cầu nối `remixd`:**
 5. Chạy lệnh đã cấu hình sẵn trong dự án:

```
bash npm run remix
```
 6. (Hoặc chạy lệnh trực tiếp bằng `npx`:

```
npx -y @remix-project/remixd -s . --remix-ide https://remix.ethereum.org
```

)
 7. **Cấp quyền hệ thống (macOS Prompt):**
 8. Nếu macOS hiện popup thông báo: *"Do you want the application node to accept incoming network connections?"* → Chọn **Allow**.
 9. **Trạng thái sẵn sàng:**
 10. Màn hình Terminal xuất hiện dòng: `[WARN] You can now connect to the remix.ethereum.org endpoint`. Giữ nguyên Terminal này chạy ngầm trong suốt buổi học.
-

Dành cho người dùng Windows:

1. **Mở Terminal trong Antigravity IDE:**
2. Dùng phím tắt: `Ctrl + `` hoặc vào menu `Terminal` → `New Terminal`.
3. **⚠ Xử lý lỗi chặn Script của PowerShell (Lỗi cực kỳ phổ biến):**
 - Nếu gõ lệnh mà Windows báo lỗi đỏ: `File ... cannot be loaded because running scripts is disabled on this system (PSecurityException)`:
 - **Cách 1 (Nhanh nhất):** Tại góc trên bên phải của khung Terminal, bấm vào dấu mũi tên v (cạnh dấu `+`) → Chọn **Command Prompt (cmd)** hoặc **Git Bash** để chạy lệnh thay vì PowerShell.

- **Cách 2 (Triệt để):** Mở cửa sổ *Windows PowerShell* bên ngoài với quyền **Run as Administrator**, gõ lệnh sau rồi gõ **Y** và Enter:

```
powershell Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

4. **Khởi chạy cầu nối** `remixd` :

5. Trong Terminal của Antigravity IDE, gõ lệnh:

```
bash npm run remix
```

6. **⚠ Xử lý tường lửa Windows Defender Firewall (BẮT BUỘC):**

7. Khi chạy lần đầu, Windows sẽ hiển thị bảng cảnh báo: *"Windows Security Alert - Windows Defender Firewall has blocked some features of Node.js"*.

8. **Thao tác quan trọng:** Tích chọn vào ô **"Private networks, such as my home or work network"** → Bấm **Allow access**.

(Nếu lỡ bấm Cancel, Remix trên trình duyệt sẽ bị tường lửa chặn và không thể đọc file từ máy tính).

9. **Trạng thái sẵn sàng:**

10. Terminal báo: `[WARN] You can now connect to the remix.ethereum.org endpoint.`

C. Kết nối trên trình duyệt Remix Web (Áp dụng chung cho cả Win & Mac)

1. Mở trình duyệt (Khuyên dùng **Chrome**, **Edge**, hoặc **Brave**), truy cập:

<https://remix.ethereum.org>.

Lưu ý cho người dùng trình duyệt Brave: Nếu không kết nối được, click vào biểu tượng đầu sư tử (Brave Shields) trên thanh URL → Tạm thời gạt tắt Shields để cho phép kết nối WebSocket nội bộ (`localhost:65520`).

2. Tại cột bên trái giao diện Remix, click vào menu xổ xuống **Workspaces** (ở góc trên cùng của File Explorer).
3. Chọn mục: **- connect to localhost -**.
4. Hộp thoại xác nhận hiện ra với nội dung cảnh báo chia sẻ thư mục → Bấm nút xanh **Connect**.
5. Nhìn vào danh sách file trên Remix: Một Workspace có tên **localhost** sẽ xuất hiện, hiển thị đầy đủ toàn bộ thư mục `contracts/` trên máy tính của bạn. Mọi thay đổi code bên Antigravity IDE sẽ lập tức đồng bộ theo thời gian thực sang Remix và ngược lại!

D. Cách đóng kết nối khi kết thúc buổi học

- Tại cửa sổ Terminal đang chạy `remixd`:
 - **macOS**: Nhấn tổ hợp phím `Control + C`.
 - **Windows**: Nhấn tổ hợp phím `Ctrl + C` (nếu terminal cmd hỏi `Terminate batch job (Y/N)?`, gõ `Y` rồi Enter).
-

Cách 2: Cài đặt ứng dụng Remix Desktop (Standalone App)

Nếu sinh viên không muốn sử dụng qua trình duyệt và muốn một phần mềm độc lập:

1. Truy cập trang chính thức của Ethereum Foundation:
<https://github.com/ethereum/remix-desktop/releases>.

2. Tải bản cài đặt tương ứng:

* **macOS**: File `.dmg` (Máy chip Apple M1/M2/M3/M4 chọn bản `arm64`; máy Mac Intel cũ chọn bản `x64`).

* **Windows**: File `.exe` (`Remix-Desktop-Setup-....exe`).

3. Cài đặt và mở ứng dụng để sử dụng như một phần mềm độc lập.

PHẦN 3: SỬ DỤNG ANTIGRAVITY IDE VIẾT SMART CONTRACT ĐẦU TIÊN

3.1. Thiết lập Extension hỗ trợ Solidity trên Antigravity IDE

1. Mở Antigravity IDE → Vào tab **Extensions** (`Ctrl + Shift + X` trên Windows hoặc `Cmd + Shift + X` trên Mac).
2. Tìm kiếm và cài đặt tiện ích:
3. **Solidity** (khuyến dùng bản của *Nomic Foundation* hoặc *Juan Blanco*).
4. Tiện ích này sẽ cung cấp tính năng tô màu cú pháp (Syntax Highlighting), tự động gợi ý code (IntelliSense) và kiểm tra lỗi cú pháp sớm.

3.2. Cấu trúc thư mục chuẩn mực sơ phạm

Dự án được tổ chức theo tiêu chuẩn module hóa:

```
crypto-smart-contract-2026/
├── contracts/
│   ├── 01_Basics/
│   │   ├── 00_HelloWorld.sol    ← File thực hành đầu tiên
│   │   ├── 01_SimpleStorage.sol ← Bài học về kiểu dữ liệu & storage
│   │   └── 02_Counter.sol       ← Bài học về gas & unchecked
│   ├── package.json
│   ├── GEMINI.md                ← Quy chuẩn hệ thống & sơ phạm
│   └── HUONG-DAN.md             ← Tài liệu hướng dẫn này
```

3.3. Phân tích cấu trúc mã nguồn: 00_HelloWorld.sol

Mở file [contracts/01_Basics/00_HelloWorld.sol](#). Dưới đây là phân tích chi tiết từng khối lệnh:

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/// @title Hello World Smart Contract
/// @author Dr. Long Ha
/// @notice Hợp đồng đầu tiên giúp sinh viên làm quen với kiến trúc cơ sở của
contract HelloWorld {
    // -----
    // 1. Biến trạng thái (State Variables)
    // Lưu vĩnh viễn trong không gian lưu trữ (Storage) của EVM.
    // Quy ước: Biến storage nên có tiền tố "s_" để dễ phân biệt chi phí Gas.
    // -----
    string private s_greeting;
    uint256 public s_updateCount;

    // -----
    // 2. Sự kiện (Events)
    // Dùng để ghi log vào khối blockchain, tốn ít gas hơn lưu trữ biến.
    // Các ứng dụng giao diện (Frontend Web3) sẽ "lắng nghe" sự kiện này.
    // -----
    event GreetingChanged(
        address indexed updater,
        string oldGreeting,
        string newGreeting,
        uint256 timestamp
    );

    // -----
    // 3. Hàm khởi tạo (Constructor)
    // Chỉ được gọi DUY NHẤT 1 LẦN khi hợp đồng được Deploy lên mạng.
    // -----
    constructor(string memory _initialGreeting) {
        s_greeting = _initialGreeting;
        s_updateCount = 0;
    }

    // -----
    // 4. Hàm Ghi (Write Function): Làm thay đổi trạng thái của Blockchain
    // → Bắt buộc phải có người ký xác nhận, tốn Gas và sinh ra Transaction.
    // -----
    function setGreeting(string calldata _newGreeting) external {
        string memory oldGreeting = s_greeting;
        s_greeting = _newGreeting;

        unchecked {
            ++s_updateCount; // Tối ưu gas cho phép tăng biến đếm
        }

        emit GreetingChanged(msg.sender, oldGreeting, _newGreeting, block.timestamp);
    }

    // -----
    // 5. Hàm Đọc (Read Function - có từ khóa "view")
    // → Chỉ đọc dữ liệu từ Storage, KHÔNG làm biến đổi trạng thái.
    // → KHÔNG tốn phí Gas khi người dùng gọi từ bên ngoài.
    // -----
    function getGreeting() external view returns (string memory) {
        return s_greeting;
    }
}

```

```
}  
}
```

PHẦN 4: KẾT NỐI ANTIGRAVITY VỚI REMIX, COMPILE & DEPLOY

Bước 1: Khởi động cầu nối `remixd` từ Antigravity IDE

Trong cửa sổ Terminal của Antigravity IDE, chạy lệnh:

```
npm run remix
```

Màn hình terminal sẽ thông báo: `[WARN] You can now connect to the remix.ethereum.org endpoint`. Cầu nối đã sẵn sàng lắng nghe ở cổng cục bộ `ws://127.0.0.1:65520`.

Bước 2: Kết nối Remix IDE với thư mục máy tính

1. Truy cập <https://remix.ethereum.org> trên trình duyệt.
2. Tại cột bên trái, click vào menu **Workspaces** (nút sổ xuống ở góc trên cùng).
3. Chọn **connect to localhost**.
4. Một bảng thông báo bảo mật hiện ra → Click **Connect**.
5. Thư mục `contracts/` trên máy của bạn lập tức xuất hiện đồng bộ trong giao diện Remix! Mọi chỉnh sửa ở Antigravity IDE hoặc Remix sẽ được lưu thẳng vào ổ cứng của bạn.

```
[Antigravity IDE] <== (remixd bridge) ==> [Remix Browser IDE]  
(Chỉnh sửa code)                        (Compile, Deploy, Debug)
```

Bước 3: Biên dịch Smart Contract (Compile)

1. Mở file `contracts/01_Basics/00_HelloWorld.sol`.
2. Click biểu tượng **Solidity Compiler** (Icon thứ 2 từ trên xuống) hoặc nhấn phím tắt `Cmd + S` (Mac) / `Ctrl + S` (Windows).
3. Chọn phiên bản: **0.8.26**.
4. Khi thấy **dấu tích xanh lá (✓)** xuất hiện bên cạnh biểu tượng Compiler → Quá trình biên dịch thành công!

5. *Thực hành quan sát*: Cuộn xuống cuối thanh Compiler, click vào nút **Compilation Details** để xem chi tiết cấu trúc **Bytecode** và **ABI**.

Bước 4: Triển khai thử nghiệm trên Remix VM (Sandbox Memory)

- Click vào biểu tượng **Deploy & Run Transactions** (Icon thứ 3).
- Kiểm tra các mục:
- ENVIRONMENT**: Chọn `Remix VM (Cancun)`. Môi trường này cấp cho bạn 10 tài khoản thử nghiệm với 100 ETH sẵn có.
- CONTRACT**: Chọn đúng hợp đồng `HelloWorld - contracts/01_Basics/00_HelloWorld.sol`.
- Deploy Parameters**: Do hợp đồng có hàm `constructor(string memory _initialGreeting)`, bạn sẽ thấy một ô nhập liệu bên cạnh nút Deploy. Nhập vào một lời chào, ví dụ: `"Xin chào Hue University!"`.
- Click nút màu cam **Deploy**.
- Quan sát cửa sổ Terminal phía dưới: Xuất hiện thông báo giao dịch thành công (màu xanh), kèm theo địa chỉ hợp đồng đã được tạo (Contract Address) và lượng Gas đã tiêu thụ.

Bước 5: Tương tác kiểm thử hàm trên Remix

Nhìn xuống mục **Deployed Contracts** ở góc dưới bên trái, bấm mở rộng hợp đồng `HelloWorld`:

- * **Nút xanh dương (`getGreeting`, `s_updateCount`)**: Click vào `getGreeting` → Trả về kết quả `"Xin chào Hue University!"` ngay lập tức, không tốn gas.
- * **Nút màu cam (`setGreeting`)**: Nhập giá trị mới vào ô text, ví dụ: `"Hello Web3 2026"` → Click nút cam → Một Transaction được tạo ra, trừ đi một lượng nhỏ Gas từ tài khoản thử nghiệm.
- * Bấm lại nút xanh `getGreeting` → Kết quả đã chuyển thành `"Hello Web3 2026"`. Bấm `s_updateCount` → Kết quả hiển thị `1`.

Bước 6: Triển khai lên mạng công khai (Sepolia Testnet)

Khi sinh viên đã sẵn sàng đưa hợp đồng lên mạng thực tế:

- Tại mục **ENVIRONMENT**, thay vì chọn *Remix VM*, hãy chọn **Injected Provider - MetaMask**.
- Cửa sổ MetaMask sẽ tự động bật lên yêu cầu cấp quyền kết nối → Bấm **Confirm**.
- Đảm bảo ví MetaMask đang ở mạng **Sepolia** và có đủ số dư test ETH.

4. Bấm nút màu cam **Deploy** → MetaMask xuất hiện bảng xác nhận giao dịch (Gas Fee) → Bấm **Confirm**.
5. Sau vài giây khi khối được xác thực trên mạng Ethereum Sepolia, hợp đồng đã chính thức tồn tại vĩnh viễn trên Blockchain! Sinh viên có thể sao chép địa chỉ hợp đồng và tra cứu trực tiếp trên <https://sepolia.etherscan.io/>.

PHẦN 5: BÀI TẬP THỰC HÀNH & CÂU HỎI TỰ ĐÁNH GIÁ

Câu hỏi trắc nghiệm tự đánh giá:

1. **Câu 1:** Vì sao không thể gọi hàm ghi (`setGreeting`) mà không mất phí Gas?
2. A. Do máy chủ của Remix thu phí.
3. B. Vì việc thay đổi biến trạng thái đòi hỏi toàn bộ các node trên mạng EVM phải tính toán và lưu trữ dữ liệu mới vào khối, phí Gas là chi phí trả cho tài nguyên tính toán này.
4. C. Do MetaMask tự động tính phí dịch vụ.
5. **Câu 2:** Nếu bạn vô tình đẩy Private Key hoặc Seed Phrase lên một GitHub Public Repository, hành động nào là ĐÚNG nhất?
6. A. Xóa commit đó trên GitHub là an toàn.
7. B. Coi như ví đó đã bị lộ vĩnh viễn, lập tức tạo một ví mới hoàn toàn và không bao giờ sử dụng lại ví cũ.
8. **Câu 3:** ABI có vai trò gì trong ứng dụng phi tập trung (DApp)?

Bài tập thực hành nâng cao (Hands-on Challenge):

- **Yêu cầu:** Mở rộng hợp đồng `HelloWorld.sol` bằng cách thêm:
- Một biến lưu địa chỉ người tạo hợp đồng: `address public immutable i_owner;`
- Ràng buộc bảo mật: Chỉ cho phép người tạo hợp đồng (`i_owner`) mới có quyền gọi hàm `setGreeting`, nếu người khác gọi thì giao dịch sẽ bị Revert với custom error: `error Unauthorized();`
- **Gợi ý kiểm thử:** Thử đổi tài khoản trên Remix sang Account thứ 2 và gọi hàm `setGreeting` để quan sát giao dịch bị chặn lại như thế nào.